
SGLang: 高效执行 结构化语言模型程序

Lianmin Zheng^{2*} Liangsheng Yin³ Zhiqiang Xie¹ Chuyue Sun¹ Jeff Huang⁴
Cody Hao Yu⁵ Shiyi Cao² Christos Kozyrakis¹ Ion Stoica² Joseph E. Gonzalez²
Clark Barrett¹ Ying Sheng^{1*}

¹ Stanford University ² UC Berkeley ³ Shanghai Jiao Tong University
⁴ Texas A & M University ⁵ Independent Researcher

Abstract

大型语言模型 (LLM) 越来越多地用于需要多次生成调用、高级提示技术、控制流以及结构化输入/输出的复杂任务。然而，目前缺乏高效的系统来编程和执行这些应用程序。我们推出了 SGLang，一个用于高效执行复杂语言模型程序的系统。SGLang 由前端语言和运行时组成。前端语言通过用于生成和并行控制的原语简化了编程。运行时通过诸如用于重用键值缓存的 Radix Attention 和用于更快结构化输出解码的压缩有限状态机等创新优化来加速执行。实验表明，在包括智能体控制、逻辑推理、少样本学习基准测试、JSON 解码、检索增强生成流水线和多轮聊天等任务上，SGLang 的吞吐量比各种大型语言和多模态模型的现有最佳推理系统高出至多 6.4 $\times$ 。代码已公开，可在 <https://github.com/sgl-project/sglang> 获取。

1 引言

近年来，逻辑学习模型 (LLM) 的性能不断提升，其应用范围也随之扩大，使其能够处理更广泛的通用任务，并作为自主代理运行。在这些应用中，LLM 会进行多轮规划、推理以及与外部环境的交互。这可以通过工具的使用、多种输入方式以及各种提示技术来实现，例如少样本学习、自洽性、思维骨架和思维树。所有这些新的应用场景都需要多次调用 LLM 生成模型，而且这些调用往往相互依赖，这表明使用多调用结构来完成复杂任务已成为一种趋势。

这些模式的出现标志着我们与语言模型 (LLM) 的交互方式发生了转变，从简单的聊天转向了更复杂的 LLM 程序化使用，这意味着使用程序来调度和控制 LLM 的生成过程。我们将这些程序称为“语言模型程序” (LM 程序)。上文提到的高级提示技术和代理工作流都属于 LM 程序的范畴。LM 程序有两个共同特征：(1) LM 程序通常包含多个 LLM 调用，并穿插着控制流。这对于完成复杂任务和提高整体质量至关重要。(2) LM 程序接收结构化输入并生成结构化输出。这对于 LM 程序的组合以及将 LM 程序集成到现有软件系统中至关重要。

尽管 LM 程序已被广泛应用，但目前用于表达和执行它们的系统效率仍然低下。我们发现高效使用语言模型 (LM) 程序面临两大主要挑战：首先，由于语言学习模型 (LLM) 的非确定性，编写 LM 程序既繁琐又困难。开发 LM 程序通常需要大量的字符串操作、对提示进行实验性调整、脆弱的输出解析、处理多种输入模态以及实现并行机制。这种复杂性显著降低了即使是简单程序的可读性 (Sec. 2)。

其次，也是非常重要的一点，由于冗余的计算和内存使用，执行 LM 程序的效率低下。目前最先进的推理引擎（例如 vLLM [19]、TGI [12] 和 TensorRT-LLM [29]）在不直接了解工作负载的情况下，已经过优化以降低延迟并提高吞吐量。这使得这些系统具有通用性和鲁

*Equal contribution.

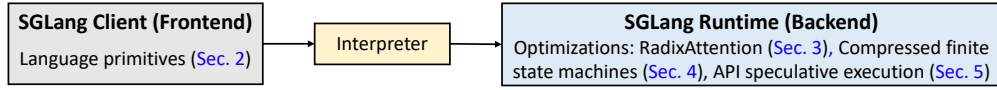


Figure 1: 系统架构：解释器以优化的运行时执行语言原语。

棒性，但也导致其在任何给定工作负载下效率显著降低。一个突出的例子是键值 (KV) 缓存的重用 (Sec. 3)。KV 缓存包含可重用的中间张量，这些张量对于生成式推理至关重要。在典型的语言模型 (LM) 程序批量执行过程中，存在许多机会在多个共享公共前缀的不同语言学习模型 (LLM) 调用之间重用 KV 缓存。然而，当前的系统缺乏有效的机制来促进这种重用，导致不必要的计算和内存浪费。另一个例子是结构化输出（例如 JSON 模式）的约束解码，其中 LLM 的输出被限制为遵循由正则表达式定义的特定语法规则 (Sec. 4)。在这些约束下，通常可以解码多个词元一次。然而，现有系统一次只能解码一个词元，导致解码速度不够理想。

为了应对这些挑战，我们提出了 SGLang，一种基于 S 结构化的 G 生成 Lang 语言，用于语言学习模型 (LLM)。其核心思想是系统地利用语言学习程序中的多调用结构来实现高效执行。如图 1 所示，它由两部分组成：前端语言和后端运行时。前端简化了语言学习程序的编程，而运行时则加速了程序的执行。这两部分可以协同工作以获得更好的性能，也可以独立运行。

我们引入了 SGLang，它是一种嵌入在 Python 中的领域特定语言。它提供了用于生成（例如 `extend`、`gen`、`select`）和并行控制（例如 `fork`、`join`）的基本功能。SGLang 与 Python 的控制流和库兼容，因此用户可以使用原生 Python 语法轻松开发高级提示工作流。我们为 SGLang 提供了一个解释器和一个编译器。解释器将提示状态作为流进行管理，并将基本操作提交到该流中进行异步执行，从而确保对同步和程序内并行性进行适当的控制。此外，SGLang 程序可以进行跟踪和编译，以进行更多优化。

在运行时方面，我们提出了几项新颖的优化措施来加速 SGLang 程序的执行。第一项技术 RadixAttention 实现了在多次生成调用之间自动重用键值缓存。在现有的推理引擎中，请求的键值缓存会在处理完成后被丢弃，这阻止了键值缓存在多次调用之间的重用，并显著降低了执行速度。而我们的系统则为基数树中的所有请求维护一个最近最少使用 (LRU) 的键值缓存。这种方法将键值缓存作为传统缓存进行管理，并使用基数树进行高效的匹配、插入和驱逐。它允许运行时使用缓存感知调度策略高效地处理各种重用模式。第二种技术是压缩有限状态机，它能够更快地对结构化输出进行约束解码。现有系统仅通过屏蔽不允许的词元的概率来遵循下一个词元的约束，因此一次只能解码一个词元。而我们的系统分析约束并构建一个压缩有限状态机来表示这些约束。这种方法尽可能地将多词元路径压缩成单步路径，从而允许一次解码多个词元，以实现更快的解码速度。最后，SGLang 还支持仅提供 API 的模型，例如 OpenAI 的 GPT-4。我们引入了第三种技术——API 推测执行，以优化仅提供 API 模型的多调用程序。

我们使用 SGLang 实现了多种 LLM 应用，包括智能体控制、逻辑推理、少样本学习基准测试、JSON 解码、检索增强生成流水线、多轮对话和多模态处理。我们在 NVIDIA A10G 和 A100 GPU 上测试了 Llama-7B/70B ([39])、Mistral-8x7B ([13])、LLaVA-v1.5-7B (图像) ([24]) 和 LLaVA-NeXT-34B (视频) ([50]) 等模型的性能。实验结果表明，与现有的编程和推理系统（包括 Guidance ([9])、vLLM ([19]) 和 LMQL ([3])）相比，SGLang 在各种工作负载、模型和硬件配置下均实现了高达 6.4× 的吞吐量提升。

2 编程模型

本节通过一个运行示例介绍 SGLang 编程模型，描述其语言原语和执行模式，并概述运行时优化机会。该编程模型通过提供灵活且可组合的原语，可以简化多调用工作流程中的繁琐操作（例如，字符串操作、API 调用、约束规范、并行处理）。

一个运行示例。该语言是一种嵌入在 Python 中的领域特定语言。Fig. 2 展示了一个使用分支求解合并提示方法 ([33]) 评估关于图像的文章的程序。函数 `multi_dimensional_judge` 接受三个参数：`s`、`path` 和 `essay`。`s` 管理提示状态，`path` 是图像文件路径，`essay` 是文章文本。可以使用 `+=` 运算符将新的字符串和 SGLang 原语追加到状态 `s` 中以执行。首先，该函数将图像和文章添加到提示中。然后，它使用 `select` 检查文章是否与图像相关，并将结果存储在 `s["related"]` 中。如果相关，提示信息会被分成三个副本，分别从不同维度进行并行评估，并使用 `gen` 将结果存储在 `f["judgment"]` 中。接下来，程序会合并评估结果，生成摘要，并分配一个字母等级。最后，程序会按照正则表达式约束 `regex` 定义的模式，以 JSON 格式返回结果。SGLang 大大简化了该程序，因为如果使用类似 OpenAI API 的接口，则等效程序由于需要手动操作字符串和控制并行性，需要编写与 2.1× 相同多行的代码。

```

dimensions = ["Clarity", "Originality", "Evidence"]
@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")
    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return
    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))
    # Merge the judgments
    judgment = "\n".join(f["judgment"] for f in forks)
    # Generate a summary and a grade. Return in the JSON format.
    s += user("Provide the judgment, summary, and a letter grade")
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
        + "The grade of it is" + gen("grade"))
    schema = r'\{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+]?"}\}'
    s += user("Return in the JSON format.")
    s += assistant(gen("output", regex=schema))
    state = multi_dimensional_judge.run(...)
    print(state["output"])

```

Handle chat template and multi-modal inputs
 Select an option with the highest probability
 Fetch result; Use Python control flow
 Runtime optimization: KV Cache Reuse (Sec. 3)
 Multiple generation calls run in parallel
 Fetch generation results
 Runtime optimization: API speculative execution (Sec. 5)
 Runtime optimization: fast constrained decoding (Sec. 4)
 Run an SGLang program

Figure 2: SGLang 中多维评估的实现使用了分支求解合并提示技术。SGLang 提供的原语以红色显示。

System	Syntax	Language Primitives	Runtime Backends
LMQL	Custom	extend, gen, select	HF Transformers, llama.cpp, OpenAI
Guidance	Python	extend, gen, select, image	HF Transformers, llama.cpp, OpenAI
SGLang	Python	extend, gen, select, image, video, fork, join	SGLang Runtime (SRT), OpenAI

语言原语。SGLang 提供了用于控制提示状态、生成和并行性的原语。它们可以与 Python 语法和库一起使用。以下是这些原语：“gen”调用模型生成提示信息，并将结果存储在第一个参数中指定的变量名中。它支持“regex”参数，用于将输出约束为遵循由正则表达式定义的语法（例如，JSON 模式）。“select”调用模型从列表中选择概率最高的选项。运算符“+=”或“extend”将字符串附加到提示符。运算符“[variable_name]”获取生成结果。“fork”创建提示符状态的并行分支。“join”重新合并提示符状态。“image”和“video”接收图像和视频输入。

执行模式。执行 SGLang 程序的最简单方法是通过解释器，其中提示符被视为异步流。诸如 extend、gen 和 select 之类的基本函数会被提交到流中进行异步执行。这些非阻塞调用允许 Python 代码继续运行，而无需等待生成完成。这类似于异步启动 CUDA 内核。每个提示符都由后台线程中的流执行器管理，从而实现程序内部并行。获取生成结果的操作会阻塞，直到结果准备就绪，以确保正确的同步。此外，SGLang 程序可以编译为计算图，并使用图执行器执行，从而实现更多优化。本文默认使用解释器模式，并在 Appendix D 中讨论编译器模式的结果。SGLang 通过其自身的 SGLang 运行时 (SRT) 支持开放权重模型，以及 OpenAI 和 Anthropic 模型等 API 模型。

比较。用于低级语言模型 (LLM) 的编程系统可以分为高级系统（例如 LangChain、DSPy）和低级系统（例如 LMQL、Guidance、SGLang）。高级系统提供预定义或自动生成的提示符，例如 DSPy 的提示符优化器。低级系统通常不修改提示符，但允许直接操作提示符和基本元素。SGLang 是一个类似于 LMQL 和 Guidance 的低级系统。Table 1 对它们的特性进行了比较。SGLang 更注重运行时效率，并带有其自身协同设计的运行时，允许后续引入新的优化。高级语言（例如 DSPy）可以编译成低级语言（例如 SGLang）。我们在 Sec. 6 中演示了如何将 SGLang 作为后端集成到 DSPy 中，以提高运行时效率。

运行时优化。Fig. 2 展示了三种运行时优化机会：键值缓存重用、快速约束解码和 API 推测执行。我们将在以下章节中讨论它们。

3 使用 RadixAttention 实现高效的键值缓存重用。

SGLang 程序可以链式调用多个生成函数，并使用“fork”原语创建并行副本。此外，不同的程序实例通常共享一些公共部分（例如，系统提示符）。这些场景会在执行过程中产生许

多共享的提示符前缀，从而为重值缓存提供了大量机会。在 LLM 推理过程中，键值缓存存储前向传播的中间张量，这些张量将被重用于解码后续的词元。它们的命名源自自注意力机制中的键值对 [40]。键值缓存的计算仅依赖于前缀词元。因此，具有相同提示符前缀的请求可以重用键值缓存，从而减少冗余计算和内存使用。更多背景信息和示例请参见 [Appendix A](#)。

鉴于键值缓存的重用机会，优化 SGLang 程序的一个关键挑战是如何在多个调用和实例之间重用键值缓存。虽然一些系统探索了某些键值缓存重用案例 ([19, 46, 14, 8])，但它们通常需要手动配置，并且无法处理所有重用模式（例如，动态树结构）。因此，大多数最先进的推理系统会为每个请求重新计算键值缓存。我们将在 [Sec. 7](#) 中讨论它们的局限性以及我们提出的不同之处。

本节介绍 RadixAttention，一种用于运行时自动、系统地重用键值缓存的新技术。与现有系统在生成请求完成后丢弃键值缓存不同，我们的系统将提示和生成结果的缓存保留在基数树中，从而实现高效的前缀搜索、重用、插入和驱逐。我们实现了 LRU 驱逐策略和缓存感知调度策略来提高缓存命中率。RadixAttention 与连续批处理 ([48])、分页注意力 ([19]) 和张量并行 ([36]) 等技术兼容。此外，在没有缓存命中的情况下，它只会引入可忽略不计的内存和时间开销。

RadixAttention。基数树是一种数据结构，它是经典 trie（前缀树）的一种空间高效替代方案。与典型的树不同，基数树的边不仅可以用单个元素标记，还可以用不同长度的元素序列标记，从而显著提高效率。在我们的系统中，我们利用基数树来管理标记序列与其对应的 KV 缓存张量之间的映射。这些 KV 缓存张量存储在非连续的分页布局中，其中每个页面的大小相当于一个标记。由于 KV 缓存会迅速填满 GPU 内存，我们引入了一种简单的 LRU 驱逐策略，优先驱逐最近最少使用的叶子节点。通过优先驱逐叶子节点，我们可以重用它们的共同祖先节点，直到这些祖先节点也成为叶子节点并被驱逐为止。

在连续批处理环境中，我们无法驱逐当前正在运行的批处理所用的节点。因此，每个节点都维护一个引用计数器，指示有多少正在运行的请求正在使用它。如果节点的引用计数器为零，则该节点可以被驱逐。请注意，我们不会预先分配固定大小的内存池作为缓存。相反，我们让缓存的令牌和当前正在运行的请求共享同一个内存池。因此，系统会动态地为缓存和正在运行的请求分配内存。当有足够多的等待请求运行时，系统会驱逐所有缓存的令牌，以便使用更大的批处理大小。[Fig. 3](#) 展示了如何为多个传入请求维护基数树。前端解释器向运行时发送完整的提示，运行时执行前缀匹配和重用。树结构存储在 CPU 上，维护开销可以忽略不计。在执行 fork 原语期间，前端首先发送前缀作为提示，以确保前缀正确插入到树中。然后，它发送剩余的提示。这种“前端提示”简化了运行时调度和匹配，体现了前端运行时协同设计的优势。

缓存感知调度。我们将缓存命中率定义为 $\frac{\text{number of cached prompt tokens}}{\text{number of prompt tokens}}$ 。当等待队列中有很多请求时，它们的执行顺序会显著影响缓存命中率。例如，如果请求调度器频繁地在不同的、不相关的请求之间切换，则可能导致缓存抖动和命中率降低。我们设计了一种缓存感知调度算法来提高缓存命中率。在批处理场景下，我们按匹配前缀长度对请求进行排序，并优先处理匹配前缀较长的请求，而不是采用先到先服务 (FCS) 的调度方式。[Alg. 1](#)（附录）展示了具有连续批处理的缓存感知调度伪代码。该算法采用最长共享前缀优先的顺序。在对延迟更敏感的场景下，我们或许仍然可以容忍有限的批处理重排序，以提高缓存重用率。此外，我们还证明了以下关于离线情况下最优调度的定理。²

Theorem 3.1. *For a batch of requests, we can achieve an optimal cache hit rate by visiting the radix tree of the requests in the depth-first search order, with a cache size $\geq$ the maximum request length. The longest-shared-prefix-first order is equivalent to a depth-first search order.*

证明见 [Sec. A.3](#)（附录）。在线情况下，深度优先搜索 (DFS) 的顺序会被打乱，但我们的调度方案仍然能够近似地模拟完整基数树增广部分的 DFS 行为，如 [Sec. A.3](#) 中所述。虽然贪婪的缓存感知调度可以实现高吞吐量，但它也可能导致资源饥饿。我们将其与其他公平调度方法的集成留作未来的工作 ([34])。

分布式场景。RadixAttention 可以扩展到多个 GPU。对于张量并行，每个 GPU 都维护一个分片的键值缓存。由于树操作相同，因此无需额外的同步。关于多工作线程的数据并行，请参阅 [Sec. A.4](#)（附录）。

²实际上，计算过程与 [Theorem 3.1](#) 证明中描述的并不相同，因为输出标记数量的不可预测性可能导致键值缓存的重新计算。

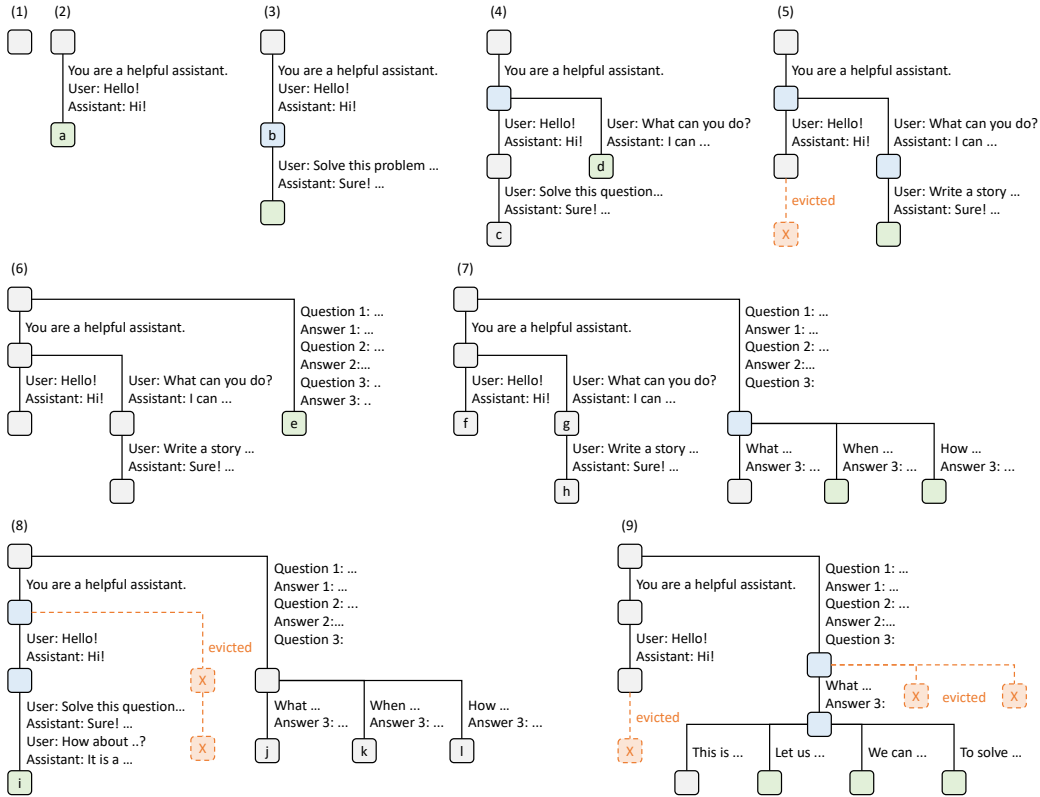


Figure 3: 图示展示了采用 LRU 驱逐策略的 RadixAttention 操作示例，涵盖九个时间点。图中展示了基数树响应各种请求的动态演化过程。这些请求包括两次聊天会话、一批少样本学习查询以及一次自洽采样。树的每条边都带有一个标签，用于表示一个子字符串或一个标记序列。节点采用颜色编码来反映不同的状态：绿色表示新添加的节点，蓝色表示该时间点访问的缓存节点，红色表示已被驱逐的节点。在步骤 (1) 中，基数树初始为空。在步骤 (2) 中，服务器处理传入的用户消息 “Hello”，并返回 LLM 输出 “Hi”。系统提示 “You are a help help assistant”、用户消息 “Hello!” 和 LLM 回复 “Hi!” 被合并到树中，形成一条连接到新节点的边。在步骤 (3) 中，服务器收到新的提示，并在基数树中找到该提示的前缀（即对话的第一轮），并重用其键值缓存。新一轮对话作为新节点添加到树中。在步骤 (4) 中，一个新的聊天会话开始。步骤 (3) 中的节点 “b” 被拆分为两个节点，以便两个聊天会话共享系统提示。在步骤 (5) 中，第二个聊天会话继续进行。然而，由于内存限制，步骤 (4) 中的节点 “c” 必须被移除。新一轮对话被添加到步骤 (4) 中节点 “d” 之后。在步骤 (6) 中，服务器收到一个少样本学习查询，对其进行处理，并将其插入到树中。由于新查询与现有节点没有任何前缀，因此根节点被拆分。在步骤 (7) 中，服务器收到一批额外的少样本学习查询。这些查询共享同一组少样本示例，因此我们将节点 “e” 从 (6) 中拆分出来以实现共享。在步骤 (8) 中，服务器收到来自第一个聊天会话的新消息。它将第二个聊天会话中的所有节点（节点 “g” 和 “h”）移除，因为它们是最近最少使用的节点。在步骤 (9) 中，服务器收到一个请求，要求对来自 (8) 的节点 “j” 中的问题进行更多答案采样，这可能是为了提示自治性。为了给这些请求腾出空间，我们将 (8) 中的节点 “i”、“k” 和 “l” 移除。

4 基于压缩有限状态机的高效约束解码

在语言模型 (LM) 程序中，用户通常希望将模型的输出限制为特定格式，例如 JSON 模式。这可以提高可控性和鲁棒性，并使输出更易于解析。SGLang 提供了一个 `regex` 参数，可以使用正则表达式来强制执行此类约束，其表达能力足以满足许多实际场景的需求。现有系统通过将正则表达式转换为有限状态机 (FSM) [42] 来支持此功能。在解码过程中，它们维护当前的 FSM 状态，从下一个状态中检索允许的标记，并将无效标记的概率设置为零，逐个标记地进行解码。然而，当有机会一次解码多个标记时，这种逐个标记的方法效率低下。例如，如图 4 (c) 所示，图 2 中的常量序列 { "summary": " 在正常的解码过程中跨越多个标记，即使在解码时只有一个有效的下一个标记，也需要多个解码阶段。因此，整个序列可以一步解码（即前向传播）。然而，现有系统一次只能解码一个词元，因为现有系统中有限状态机 (FSM) 和模型运行器之间缺乏集成，导致无法处理多个词元，从而造成解码速度缓慢。

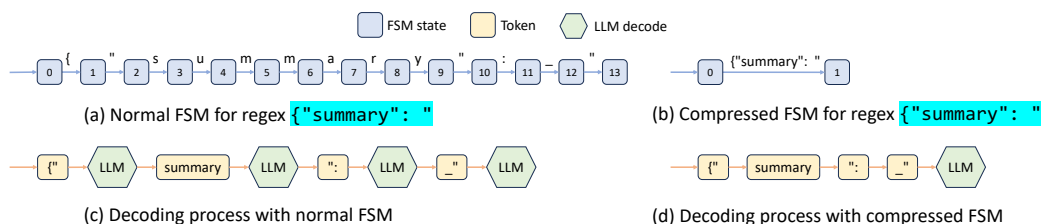


Figure 4: 普通和压缩有限状态机的解码过程（下划线 _ 表示空格）。

SGLang 通过创建一个快速的、具有压缩有限状态机的约束解码运行时来克服这一限制。该运行时分析有限状态机，并将有限状态机中相邻的奇异转换边压缩成单个边，如图 4 (b) 所示，从而能够识别何时可以同时解码多个词元。在 Fig. 4 (d) 中，压缩转换边上的多个词元可以在一次前向传播中解码，这大大加快了解码过程。它也具有通用性，适用于所有正则表达式。更多背景和实现细节请参见 Appendix B。

5 使用 API 推测执行实现高效的端点调用

前面的章节介绍了针对开放权重模型的优化，这些优化需要修改模型推理过程。此外，SGLang 也适用于仅支持 API 访问的模型，例如 OpenAI 的 GPT-4。但是，对于这些模型，我们只能调用黑盒 API 端点。

本节介绍了一种针对黑盒 API 模型的新优化，它利用推测执行来加速执行，并降低多次调用 SGLang 程序的 API 开销。例如，程序可能要求模型生成一个字符的描述，其调用模式为：`s += context + "name:" + gen("name", stop="\n") + "job:" + gen("job", stop="\n")`。简单来说，两个 `gen` 原语对应两次 API 调用，这意味着用户需要为 `context` 的输入令牌支付两次费用。在 SGLang 中，我们可以在第一次调用时启用推测执行，并使其忽略停止条件，继续生成更多令牌。解释器会保留这些额外的生成输出，并将其与后续原语进行匹配和重用。在某些情况下，通过精心的提示工程，模型可以高精度地匹配模板，从而节省一次 API 调用的延迟和输入成本。

6 评估

我们评估了 SGLang 在各种 LLM 工作负载下的性能。随后，我们进行了消融研究和案例研究，以展示特定组件的有效性。SGLang 使用 PyTorch ([31]) 实现，并采用了来自 FlashInfer ([47]) 和 Triton ([38]) 的自定义 CUDA 内核。

6.1 设置

模型。我们测试了密集 Llama-2 模型 ([39])、稀疏混合专家模型 Mixtral ([13])、多模态 LLaVA 图像模型 ([23]) 和视频模型 ([50])，以及 OpenAI 的 GPT-3.5 API 模型。对于开放权重模型，参数数量在 70 亿到 700 亿之间，我们使用 float16 精度。

硬件。我们的大部分实验都在配备 NVIDIA A10G GPU (24GB) 的 AWS EC2 G5 实例上运行。我们在单个 A10G GPU 上运行 7B 模型，并在多个 A10G GPU 上运行更大的模型（利用张量并行 [36]）。我们还在 A100G (80GB) GPU 上运行了一些额外的实验。

基准测试。我们将 SGLang 与使用各自语言和默认运行时环境的高级编程系统，以及使用标准 OpenAI 式补全 API 的底层推理引擎进行比较。除非另有说明，否则我们不会启用任何会改变计算结果的优化，以确保所有系统计算出相同的结果。基准测试包括：

- Guidance ([9])，一种用于控制 LLM 的语言。我们使用 Guidance v0.1.8 版本，后端为 llama.cpp。
- vLLM ([19])，一个高吞吐量推理引擎。我们使用 vLLM v0.2.5 版本及其默认 API 服务器³。
- LMQL ([3])，一种查询语言。我们使用 LMQL v0.7.3 版本，后端为 Hugging Face Transformers。

工作负载。我们测试了以下内容：5-shot MMLU ([10]) 和 20-shot HellaSwag ([49]) 基准测试。对于 MMLU，我们解码一个词元；对于 HellaSwag，我们使用原始算法 (`select`)

³RadixAttention 已作为可选的实验性功能部分集成到最新版本的 vLLM 中；因此，我们使用早期版本进行比较。

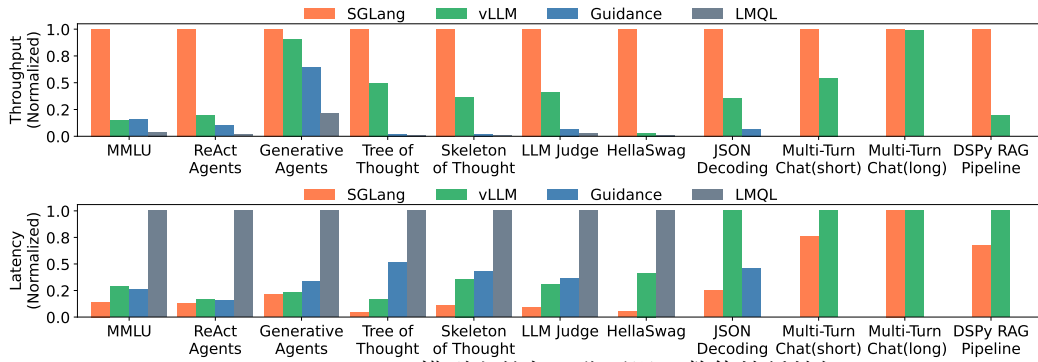


Figure 6: Llama-7B 模型上的归一化延迟。数值越低越好。

选择概率最高的答案。对于 ReAct 智能体 ([45]) 和生成式智能体 ([30])，我们从原始论文中提取轨迹并进行重放。对于 GSM-8K 问题，我们使用 Tree-of-thought ([44])，对于 Skeleton-of-thought ([28]) 生成提示。我们使用 LLM 评判器，并采用 branch-solve-merge ([33]) 技术；使用正则表达式指定的模式进行 JSON 解码；以及包含 4 个回合的多轮聊天，其中每个回合的输入在 256 到 512 个词元之间随机采样。多轮聊天（短）指短输出（4-8 个词元），多轮聊天（长）指长输出（256-512 个词元）；DSPy 检索增强生成 (RAG) 流水线 [16] 在其官方示例中。

指标。我们报告两个性能指标：吞吐量和延迟。对于吞吐量，我们运行足够大的程序实例批次来计算最大吞吐量，比较每秒执行的程序实例数（每秒程序数，p/s）。对于延迟，我们一次执行一个程序，不进行批处理，并报告多个实例的平均延迟。

6.2 端到端性能

开放权重模型的结果。Fig. 5 和 Fig. 6 分别展示了延迟和吞吐量测试结果。SGLang 的吞吐量比 6.4 $\times$ 提升了至多，延迟比 3.7 $\times$ 降低了至多。这些改进得益于 KV 缓存的重用、单个程序内部并行性的利用以及更快的约束解码。接下来，我们将解释每个基准测试中速度提升的原因。

在 MMLU 上，SGLang 可以利用 RadixAttention 重用 5-shot 示例的 KV 缓存。RadixAttention 可以同时提升吞吐量和降低延迟。RadixAttention 通过共享 KV 缓存来减少总内存使用量，从而允许更大的批处理大小，进而提高最大吞吐量。RadixAttention 还减少了预填充的计算量，从而降低了首词延迟。在 HellaSwag 中，SGLang 复用了少样本示例和多选题通用前缀的键值缓存，实现了两级共享。对于 ReAct 和生成式智能体，SGLang 复用了智能体模板和先前调用的键值缓存。在 Tree-of-thought 和 Skeleton-of-thought 中，SGLang 在单个程序内并行化生成调用，并尽可能地复用键值缓存。在 JSON 解码中，SGLang 通过使用压缩的有限状态机同时解码多个标记来加速解码。在多轮聊天中，SGLang 复用了聊天历史记录的关键值缓存。对于短输出，加速效果更为显著，因为键值缓存的复用主要用于减少前缀处理时间。对于长输出，由于不同聊天会话之间的共享较少，且解码时间占主导地位，因此几乎没有加速效果。在 DSPy RAG 流水线中，SGLang 复用了通用上下文示例的键值缓存。在这些基准测试中，缓存命中率介于 50 % 到 99 % 之间。附录 Fig. 13 列出了所有测试的实际缓存命中率和最优缓存命中率，表明我们的缓存感知调度平均接近最优命中率的 96 %。

由于性能缓慢且功能缺失，我们在最后五个基准测试中排除了 LMQL 和 Guidance。LMQL 的问题源于其缓慢的标记级处理速度和未优化的后端，而 Guidance 则缺乏批处理和并行支持。

张量并行化在大模型上的结果。我们在同一组基准测试数据集上运行了更大的模型 Mixtral-8x7B 和 Llama-70B，并采用了张量并行化，结果分别在 Fig. 7 和 Fig. 12（附录）中报告。大模型的加速比趋势与小模型相似，表明我们的优化方法能够很好地推广到大模型。由于 Guidance 和 LMQL 缺乏高效的张量并行化实现，因此我们在此省略了它们。

多模态模型的结果。SGLang 通过 image 和 video 原语原生支持多模态模型。本文中的优化方法与多模态模型兼容。对于 RadixAttention，我们计算输入图像的哈希值并将其用作基数树中的键，从而可以重用同一图像中图像标记的 KV 缓存。我们在 llava-bench-in-the-wild 上运行 LLaVA-v1.5-7B（图像），在 ActivityNet 上运行 LLaVA-NeXT-34B（视频）。由于其他基线系统对这些模型的支持不佳，我们使用 Hugging Face Transformers 中模型作者的原始实现作为基线。如 Table 2 所示，在这些基准测试中，SGLang 的吞吐量比 6 $\times$ 高出 3 倍。在

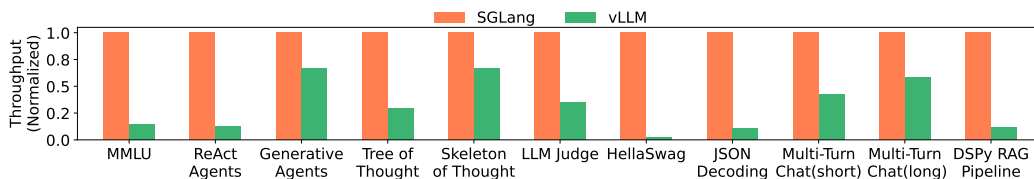


Figure 7: 基于 Mixtral-8x7B 模型吞吐量并行的对比。吞吐量数值越高越好。

Model	LLaVA-v1.5-7B (image)	LLaVA-NeXT-34B (video)
Author's original implementation	0.18 image/s	0.02 frame/s
SGLang	1.15 image/s	0.10 frame/s

llava-bench-in-the-wild 中，存在多个关于同一图像的问题，在这种情况下，SGLang 运行时会重用 KV 缓存。

生产环境部署。SGLang 已部署在 Chatbot Arena 中，[5] 用于服务开放权重模型。由于某些模型的流量较低，每个模型仅由一个 SGLang 工作进程提供服务。一个月后，我们观察到 LLaVA-Next-34B [24] 的 RadixAttention 缓存命中率为 52.4%，Vicuna-33B [4] 的缓存命中率为 74.1%。缓存命中来自常见的系统消息、频繁使用的示例图像以及多轮聊天记录。这使得 Vicuna-33B 的首词延迟平均降低了 1.7×。

API 模型的结果。我们使用 OpenAI 的 GPT-3.5 模型测试了一个从维基百科页面提取三个字段的提示。通过使用少样本提示，API 推测执行的准确率很高，并且由于提取了三个字段，输入词的成本降低了约三分之二。

6.3 消融研究

缓存命中率与延迟/吞吐量的关系。Fig. 8 (a)(b) 展示了在思维树基准测试中，缓存命中率与性能指标（首词延迟、总延迟、批处理大小和吞吐量）之间的关系。该图是通过在运行时部分禁用匹配词而获得的。它表明，更高的缓存命中率会导致更大的批处理大小、更高的吞吐量和更低的延迟。

RadixAttention 的有效性。我们在几个具有代表性的基准测试中测试了 RadixAttention 及其组件的有效性。如 Fig. 8 (c) 所示，“无缓存”表示不使用任何缓存，“无树状结构”表示使用简单的基于表的缓存而非树状结构的缓存，“FCFS 调度”表示使用先到先服务策略而非我们提出的缓存感知调度，“随机调度”表示使用随机顺序调度请求，“无前端并行”表示禁用解释器中的并行性，“无前端提示”表示禁用从解释器发送 fork 提示，“完全优化”表示启用所有优化。实验结果表明，所有这些组件对于实现最佳性能都是必需的。禁用前端解释器的并行性和提示也会导致运行时性能欠佳，这凸显了前端语言和运行时协同设计的重要性。

RadixAttention 的开销。我们在一个没有任何键值缓存重用的基准测试中测试了 RadixAttention 的开销。该基准测试在 ShareGPT 数据集上测量吞吐量。运行 100 个请求耗时 74.3 秒；然而，管理 RadixAttention 数据结构所花费的时间仅为 0.2 秒，这几乎可以忽略不计，开销小于 0.3%。这是因为树操作的复杂度是线性的，且很小。因此，我们可以默认启用 RadixAttention。

压缩有限状态机的有效性。我们在 JSON 解码基准测试中测试了压缩有限状态机及其组件的有效性。实验结果表明，压缩有限状态机能够一次性解码多个词元，从而将吞吐量提高了 1.6×。此外，我们需要对状态机进行预处理，并将其用于一批请求。否则，对每个请求都进行预处理会导致吞吐量降低 2.4×。

7 相关工作

许多工作都探索了键值缓存的重用，其中许多与我们的工作类似。独特之处在于，我们的 RadixAttention 首次提出将键值缓存视为基于树的 LRU 缓存。它是第一个支持多级共享、缓存感知调度、前端运行时协同调度和分布式场景的解决方案。vLLM [19] 和 ChunkedAttention [46] 探索了一些简单的重用案例（例如，系统提示符共享），但并未涵盖多级树状结构共享或 LRU 缓存。PromptCache [8] 提出了在前缀之外模块化重用 KV 缓存的方法，但可能会导致精度下降高达 43%。HydraGen [14]、FlashInfer [47] 和 ChunkedAttention [46]

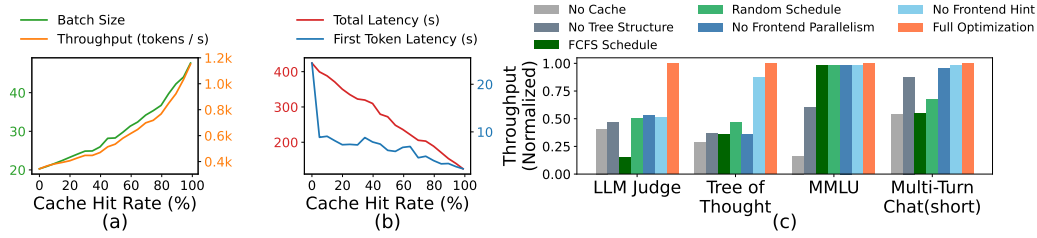


Figure 8: (a)(b) 缓存命中率消融研究。(c) RadixAttention 消融研究。

专注于 CUDA 内核优化，并未包含 LRU 缓存的概念。API Serve [1] 和 LLM-SQL [25] 研究了特定应用场景下的 KV 缓存重用，例如与外部 API 调用和关系数据库交错执行，但它们没有采用我们的基数树或缓存感知调度机制。

现有的语言学习模型 (LLM) 编程和代理框架有多种，例如 Guidance ([9])、LMQL ([3])、DSPy ([16])、LangChain ([20])、AutoGen ([43]) 和 LLM Compiler ([17])。Guidance 和 LMQL 与 SGLang 最为相似，我们在 Sec. 2 中对它们进行了比较。我们的创新之处在于针对所提出的编程模型提出了新颖的运行优化，以加速其运行。SGLang 与其他框架兼容，并且可以加速它们（例如，我们评估中的 DSPy 示例）。此外，SGLang 还与许多其他常见的推理优化 ([48, 32, 2, 19, 47, 6, 22, 11, 15, 27, 26, 7]) 兼容。

8 未来方向和结论

未来方向。尽管 SGLang 取得了进展，但仍存在一些局限性，这些局限性也为未来的研究指明了方向。这些局限性包括：扩展 SGLang 以支持更多输出模式；使 RadixAttention 能够在内存层次结构的多个层级（例如，DRAM、磁盘）上运行 ([35])；在 RadixAttention 中启用模糊语义匹配；在 SGLang 之上提供更高级别的原语；修复缓存感知调度中的饥饿问题 ([34])；以及增强 SGLang 编译器以执行高级静态优化，例如调度和内存规划。

结论：我们提出了 SGLang，一个用于高效编程和执行结构化语言模型程序的框架。SGLang 通过 RadixAttention、压缩有限状态机和语言解释器等创新优化方法，显著提高了复杂语言模型程序的吞吐量和延迟。它是开发高级提示技术和代理工作流程的宝贵工具。源代码已公开。

9

致谢 本项目由斯坦福大学自动推理中心以及 Astronomer、谷歌、IBM、英特尔、Lacework、微软、穆罕默德·本·扎耶德人工智能大学、Nexla、三星 SDS、Uber 和 VMware 的捐赠支持。郑连民获得 Meta 博士奖学金资助。感谢张元涵和李波对 LLaVA-NeXT（视频）的支持。

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. Apiserve: Efficient api support for large-language model inferencing. *arXiv preprint arXiv:2402.01869*, 2024.
- [2] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.
- [3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023.
- [4] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.
- [5] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132*, 2024.

- [6] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- [7] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. In *The Twelfth International Conference on Learning Representations*, 2023.
- [8] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.
- [9] Guidance AI. A guidance language for controlling large language models. <https://github.com/guidance-ai/guidance>. Accessed: 2023-11.
- [10] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2020.
- [11] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- [12] Hugging Face. Text generation inference. <https://github.com/huggingface/text-generation-inference>. Accessed: 2023-11.
- [13] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [14] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-throughput llm inference with shared prefixes. *arXiv preprint arXiv:2402.05099*, 2024.
- [15] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. *arXiv preprint arXiv:2403.05527*, 2024.
- [16] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [17] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2023.
- [18] Michael Kuchnik, Virginia Smith, and George Amvrosiadis. Validating large language models with relm. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [20] LangChain AI. Langchain. <https://github.com/langchain-ai/langchain>. Accessed: 2023-11.
- [21] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- [22] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.
- [23] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. *arXiv preprint arXiv:2310.03744*, 2023.
- [24] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024.

- [25] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821*, 2024.
- [26] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2024.
- [27] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- [28] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. Skeleton-of-thought: Prompting LLMs for efficient parallel generation. In *The Twelfth International Conference on Learning Representations*, 2024.
- [29] NVIDIA. Tensorrt-llm. <https://github.com/NVIDIA/TensorRT-LLM>. Accessed: 2023-11.
- [30] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *In the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*, UIST ’23, New York, NY, USA, 2023. Association for Computing Machinery.
- [31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [32] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [33] Swarnadeep Saha, Omer Levy, Asli Celikyilmaz, Mohit Bansal, Jason Weston, and Xian Li. Branch-solve-merge improves large language model evaluation and generation. *arXiv preprint arXiv:2310.15123*, 2023.
- [34] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. *arXiv preprint arXiv:2401.00588*, 2023.
- [35] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: high-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.
- [36] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [37] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for llm serving. 2024.
- [38] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [39] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [41] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2022.

- [42] Brandon T Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv e-prints*, pages arXiv–2307, 2023.
- [43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- [44] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- [45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2022.
- [46] Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.
- [47] Zihao Ye, Lequn Chen, Ruihang Lai, Yilong Zhao, Size Zheng, Junru Shao, Bohan Hou, Hongyi Jin, Yifei Zuo, Liangsheng Yin, Tianqi Chen, and Luis Ceze. Accelerating self-attentions for llm serving with flashinfer, February 2024.
- [48] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- [49] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.
- [50] Yuanhan Zhang, Bo Li, haotian Liu, Yong jae Lee, Liangke Gui, Di Fu, Jiashi Feng, Ziwei Liu, and Chunyuan Li. Llava-next: A strong zero-shot video understanding model, April 2024.

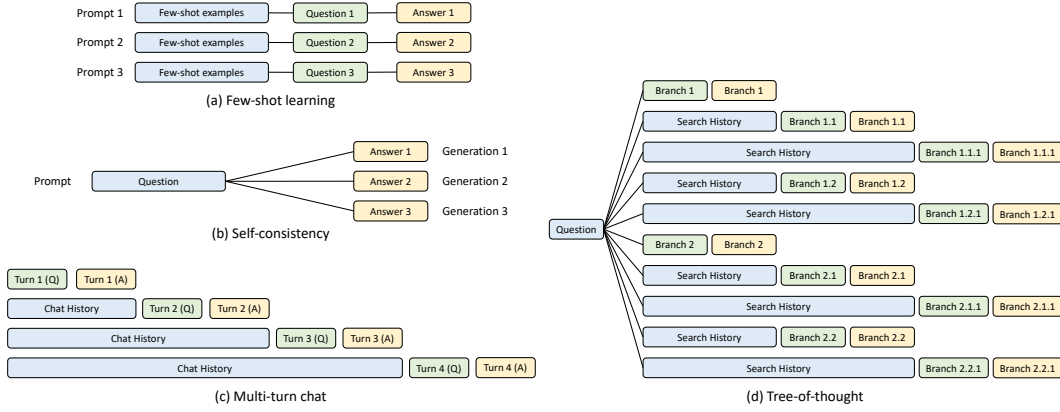


Figure 9: KV 缓存共享示例。蓝色方框表示可共享的提示部分，绿色方框表示不可共享的部分，黄色方框表示不可共享的模型输出。可共享元素包括少样本学习示例、自洽性问题 ([41])、多轮聊天中的聊天记录以及思维树中的搜索历史 ([44])。

A 关于 RadixAttention 的更多细节

A.1 关于 KV 缓存的背景

目前大多数语言学习模型 (LLM)，例如 GPT-3、PaLM 和 LLaMA，都基于自回归 Transformer 架构。这些模型根据序列中前面的词元预测下一个词元的概率。在推理过程中，模型首先通过前向传递处理输入词元序列（此过程称为“预填充”）。然后，它按顺序解码输出词元，每个词元都依赖于前面的词元（此过程称为“解码”）。我们将从输入词元序列生成输出词元序列的过程称为单次生成调用。在此过程中，每个词元都会生成一些中间张量，这些张量用于解码后续词元。这些中间张量被称为“键值缓存”，其名称来源于自注意力机制中的键值对。本文讨论优化时的一个重要观察是，KV 缓存的计算仅依赖于所有先前的标记，因此具有相同前缀的不同序列可以重用前缀标记的 KV 缓存，从而避免冗余计算。

在 LLM 程序中，通常会将多个文本段和生成调用附加到单个提示符。跨多个链式调用缓存先前标记的 KV 缓存可以减少冗余计算。然而，这种优化并非免费且简单，因为它需要额外的存储空间和更复杂的内存管理。此外，LLM 程序通常会从单个提示符生成多个输出，或者从当前状态派生一个新的提示符 [21]。vLLM [19] 研究了基本的前缀共享。还可以采用更高级的共享模式，例如不规则树状结构的共享。Fig. 9 展示了跨多个调用共享 KV 缓存的四种典型模式；现有的系统都无法自动处理所有这些问题。相反，我们在 Sec. 3 中提出的 RadixAttention 可以在运行时自动处理所有这些问题。

A.2 缓存感知调度的伪代码。

Alg. 1 展示了基于连续批处理的 RadixAttention 的缓存感知调度的伪代码。

A.3 Theorem 3.1 的证明

Theorem 3.1. *For a batch of requests, we can achieve an optimal cache hit rate by visiting the radix tree of the requests in the depth-first search order, with a cache size $\geq$ the maximum request length. The longest-shared-prefix-first order is equivalent to a depth-first search order.*

Proof. 首先，我们证明深度优先搜索 (DFS) 顺序可以达到最优缓存命中率。令 R 表示批次中的请求集合， T 表示由 R 构建的基数树。对于 T 的每条边 e ，与 e 关联的 KV 缓存至少需要计算一次。令 $|e|$ 表示与 e 关联的 KV 缓存的大小。令 C 表示 R 的 KV 缓存的计算复杂度。我们得到下界

$$C \geq \sum_{e \in \text{edges}(T)} |e|.$$

假设我们以 DFS 顺序访问基数树 T 。对于 T 的每条边 e ，首次计算与 e 关联的 KV 缓存时，我们将计算 e 的整个子树。在计算 e 的子树期间，边 e 将被持续命中，因此不会进行额外的计算。在完成以 e 为根的子树的计算后，边 e 将不再被访问。请注意，当缓存大小 $\geq$

Algorithm 1 基于连续批处理的 RadixAttention 的缓存感知调度。

Input: The radix tree T , the memory pool P , the current running batch B , the waiting queue Q .

Output: Finished requests and updated system state.

```

// Get all requests from the waiting queue
requests ← Q.get_all_requests()
// Search for the prefix matching for all waiting requests
for req ∈ requests do
    req.prefix_node, req.prefix_len ← T.match_prefix(req.input_tokens)
end for
// Sort the requests according to the matched prefix lengths
requests.sort()
// Select requests for the next batch
available_size ← T.evictable_size() + P.available_size()
current_size ← 0
new_batch ← []
for req ∈ requests do
    if req.size() + current_size < available_size then
        new_batch.append(req)
        delta ← T.increase_ref_counter(req.prefix_node)
        available_size ← available_size + delta
    end if
end for
Q.remove_requests(new_batch)
// Insert requests into the current running batch
B.merge(new_batch)
// Allocate new memory and do eviction if necessary
needed_size ← B.needed_size()
success, buffer ← P.alloc(needed_size)
if not success then
    T.evict(needed_size)
    success, buffer ← P.alloc(needed_size)
end if
B.run(buffer)
// Process finished requests
finished_requests ← B.drop_finished_requests()
for req ∈ finished_requests do
    T.decrease_ref_counter(req.prefix_node)
    T.insert(req)
end for
return finished_requests

```

于最大请求长度（即基数树 T 中的最长路径）时，边 e 在其子树的计算过程中不会被驱逐，因为包含子树中 e 的公共前缀将被持续命中。因此，与每条边 e 关联的 KV 缓存只会计算一次。由此，我们达到了下界

$$C = \sum_{e \in \text{edges}(T)} |e|.$$

缓存命中率定义为

$$\frac{\sum_{r \in R} \text{number of cached prefill tokens in } r}{\sum_{r \in R} \text{number of prefill tokens in } r},$$

等于 $1 - \frac{C}{\sum_{r \in R} \text{number of prefill tokens}}$ ，达到了其上界，从而实现了最优性。接下来，我们通过归纳法证明，最长共享前缀优先排序等价于深度优先搜索 (DFS) 排序。

- (基础) 初始时, 由于没有任何缓存, 系统会随机处理一个对应于 T 中节点 x 的请求。从根节点到 x 路径上所有对应于节点 $\{v_1, \dots, v_n\}$ 的请求无需重新计算。对应于节点 $\{v_1, \dots, v_n, x\}$ 的请求的计算复杂度与有效的深度优先搜索 (DFS) 一致。从根节点到 x 的路径已被缓存。
- (归纳) 假设我们刚刚访问了 T 中的节点 y , 并且访问的节点符合 DFS 的顺序。令 P 表示从根节点到 y 的路径。那么, 每个尚未访问的节点都与 P 中已访问的节点具有最低公共祖先。由于 P 上的节点已被缓存, 因此尚未与 P 上的最近公共祖先节点访问过的节点 z 将拥有最长共享前缀。最长共享前缀优先的排序方式会选择 z , 这是一个有效的深度优先搜索 (DFS) 顺序。从根节点到 z 的路径将被缓存, 因为它是最近的路径。

□

在在线情况下, DFS 排序会被打乱, 但最长共享前缀调度在隐含基数树的增广部分上仍然近似于 DFS 行为。我们通过考虑添加新一批请求的步骤来证明这一点。

令 T 表示目前已访问过的基数树部分, T' 表示添加新一批请求后得到的整个新基数树。令 C 表示 T 中缓存的节点集合。令 $\text{longest}(C)$ 表示 C 中从根节点到该节点的路径最长且在 T' 中有一个尚未被完全访问的子树的节点。

最长共享前缀调度算法将以深度优先搜索 (DFS) 的顺序处理 T' 中以 $\text{longest}(C)$ 为根的子树。在此过程中, 可能会发生驱逐, C 中剩余的缓存节点将变为 $C^{(1)} \subseteq C$ 。然后, 将对 T' 中以 $\text{longest}(C^{(1)})$ 为根的子树进行深度优先搜索。

类似地, 我们将得到 $C^{(2)}, \dots, C^{(k)}$, 直到 $C^{(k)}$ 中只剩下一个叶节点, 该叶节点位于 $C^{(k)}$ 中且其在 T' 中的子树尚未被完全访问。至此, 我们已达到有效的深度优先搜索 (DFS) 状态。 T' 的剩余部分将按照 Theorem 3.1 证明中所述的 DFS 顺序进行访问。

A.4 并行数据分布式基数注意力机制

为了使 RadixAttention 适应具有多个副本工作节点的分布式环境 (即数据并行), 我们开发了一种机制: 每个工作节点维护自己的子树, 而路由器则管理一个元树。该元树充当 trie 树, 跟踪所有子树及其关联的设备。当路由器收到一批新的请求时, 会在元树上执行前缀匹配。我们基于每个请求的亲性 (通过与特定工作节点以及同一组中其他请求共享的前缀长度来衡量) 实现了各种策略, 以做出高效的调度决策, 从而最大限度地减少冗余计算。每次处理新请求时, 路由器和工作节点都会独立地更新各自的树。如果工作节点发生驱逐事件, 它会将该驱逐事件提交到一个队列中, 路由器随后会在活动低谷期处理该队列以更新元树。我们使用四个工作节点和 MMLU 数据集对该分布式配置进行了基准测试, 结果表明, 这种弱一致性分布式缓存设计实现了线性扩展和最优缓存命中率, 且开销极小。最大化数据局部性和并行处理效率之间存在权衡。探索更高级的调度策略以优化这种权衡被指定为未来的研究方向。此外, Preble [37] 的并发研究基于早期 SGLang 版本的并行数据调度。

B 压缩有限状态机的更多细节

本节讨论了用于更快约束解码的压缩有限状态机的背景和实现细节。我们的目标是使语言逻辑模型 (LLM) 遵循正则表达式 (regex), 这样可以提供更强的表达能力, 并可用于表示常见的格式, 例如 JSON 模式。为了实现这一点, 我们将正则表达式转换为有限状态机 (FSM), 以指导解码过程中的生成过程。FSM 本质上是一个图, 其中包含节点 (状态) 和边 (包含字符串/字符的转换)。从初始状态开始, 每个转换都会将字符串附加到边上以移动到下一个状态, 并使用一组最终状态来结束该过程。此机制指导 LLM 的解码, 并根据 FSM 的当前状态转换过滤掉无效标记, 如图所示。解码过程可能涉及 FSM 中的多个转换, 直到达到最终状态。

约束解码的挑战在于, 约束通常以自然语言格式表达, 例如, 正则表达式由字符串/字符串表示, 而语言学习模型 (LLM) 旨在将这些字符串/字符串解释和处理为标记。这造成了复杂的场景, 因为字符串和标记之间的映射关系错综复杂, 缺乏一一对应的关系 [18]。

本节内容源自之前的博文⁴。建议读者阅读该博文以了解更多背景信息并更好地理解。

⁴<https://lmsys.org/blog/2024-02-05-compressed-fsm/>

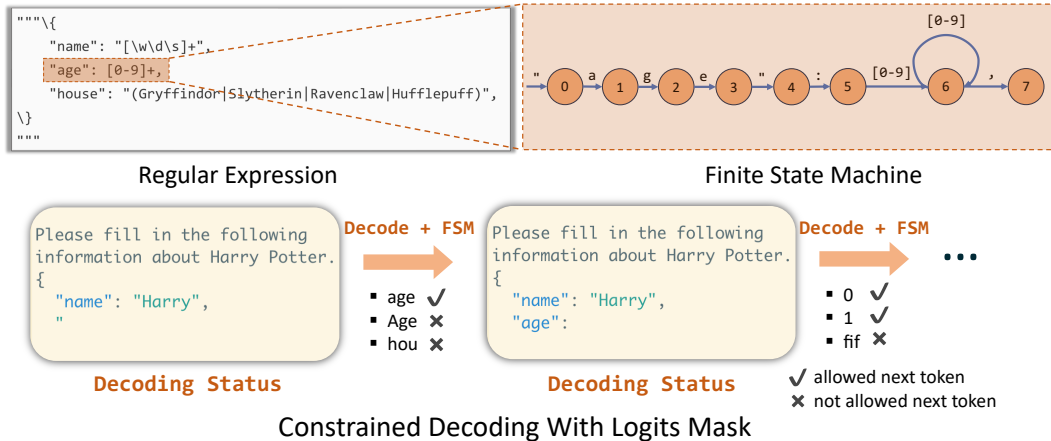


Figure 10: 示例：正则表达式如何转换为 FSM 以及 FSM 如何指导解码过程。

B.1 压缩有限状态机的实现细节

为了简化压缩有限状态机的构建，我们基于字符/字符串而非标记构建原始有限状态机。我们正式定义奇异转换边和压缩边的概念如下：

- 奇异转换边：如果一条边满足以下两个条件，则称其为奇异转换边：1) 其源节点只有一个后继节点；2) 其上只有一个可接受的字符/字符串。
- 压缩边：一条边由若干个连续相邻的边 ($e_0, e_1, \dots, e_k$) 压缩，当且仅当 $e_1, \dots, e_k$ 均为奇异转换边。压缩边的文本是 $e_0, e_1, \dots, e_k$ 中各文本的连接。

从基于字符的有限状态机 (FSM) 开始，我们递归地将奇异转换边合并到其前面的边中，直到无法进一步压缩为止，从而得到一个压缩 FSM。这种方法可以加快解码过程，如 SGLang 使用压缩 FSM 后的运行效率所示（见 Fig. 11）。

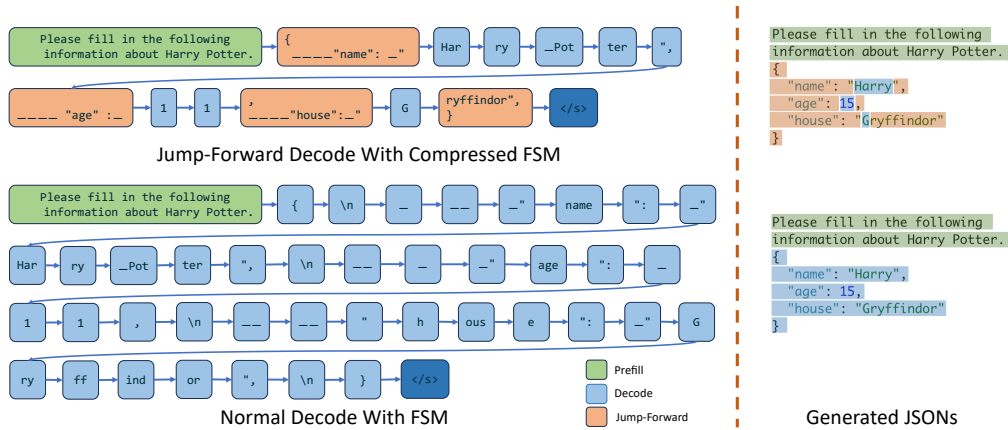


Figure 11: 压缩有限状态机 (FSM) 与普通 FSM 解码的比较：左侧子图描绘了每次前向传递的解码过程，而右侧子图解释了各种结果组件的来源。

B.2 使用重分词处理分词产物

当生成一个新的词元时，我们获取该词元的字符串，并搜索当前状态的所有出边，找到以刚刚解码的字符串开头的边，然后继续向前。然而，当转换边压缩良好且包含非常长的字符串时，我们可能也会预测下一轮的解码字符串。这就是加速发生的地方，我们将此过程称为“跳跃前向”。但是，我们仍然需要将字符串转换为词元以进行下一阶段的解码，但由于 LLM 特定的预训练和分词方法，这并非易事；直接分割可能会改变预期含义。例如，正则表达式 Fig. 2 中的压缩文本是 `{ "summary": " " }`，根据分词器，它只能被分词为 `{ " "`、`summary`、`" "` 和 `" "`，而无法像 `{ " "`、`summa`、`ry` 和 `" "` 那样随机划分。为了解决这个问题，我们使用原始分词器对所有之前的文本以及压缩边缘的文本进行重新分词，以确保与 LLM 的原始输入格式保持一致。而且，这只会带来很小的重新分词开销。

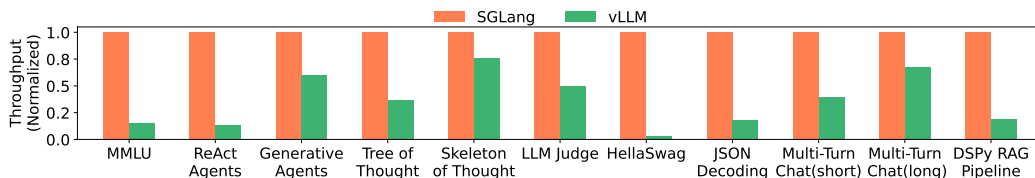


Figure 12: 启用张量并行的 Llama-2-70B 模型的归一化吞吐量。数值越高越好。

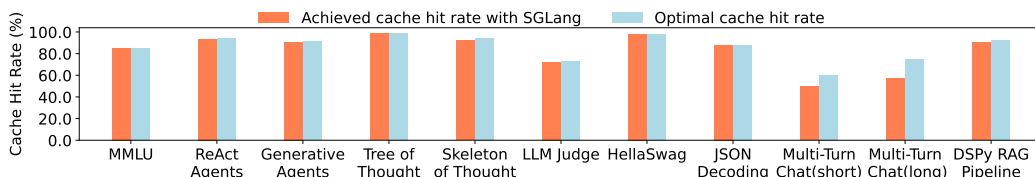


Figure 13: 各种基准测试中达到的缓存命中率和最佳缓存命中率。

B.3 未来扩展：解决概率失真问题

字符串和标记之间的差异带来的挑战也导致了概率分布偏斜的问题。例如，在 Fig. 2 中，正则表达式 "[ABCD] [+]? " 建议的等级范围是 A+ 到 D-，但如果将其替换为更宽泛的术语（例如 Excellent|Above Average|Fair|Below Average），运行时可能会错误地将 A 映射到 Above Average，因为术语 Above Average 处于压缩转换状态，从而错误地表示了等级层级。这是因为 LLM 无法识别特定的选择范围，导致生成不恰当的标记序列。计算每个选择的准确概率需要将所有导致该选择的标记序列的概率相加，这会增加解码的复杂性和开销。一种解决方法是将选择或正则表达式直接包含在预填充提示中，引导 LLM 识别其选择并以正确的标记序列输出决策。然而，这种方法并不能解决概率失真的根本问题，凸显了进一步研究以提高压缩有限状态机精度的必要性。

C 其他实验设置和结果

其他实验设置。Fig. 5 和 Fig. 6 是通过在单个 A10G (24GB) GPU 上运行 Llama-7B 获得的。Fig. 7 是通过在 8 个 A10G (24GB) GPU 上运行 Mixtral-8x7B 并启用张量并行获得的。Fig. 8 (c) 是通过在单个 A10G (24GB) GPU 上运行 Llama-7B 获得的。Fig. 12 是通过在 4 个 A100G (80GB) GPU 上运行 Llama-70B 并启用张量并行获得的。Table 2 是通过在单个 A10G (24GB) GPU 上运行 LLaVA-v1.5-7B 以及在单个 A100G (80GB) GPU 上运行 LLaVA-Next-34B 获得的。基准测试图中的每个柱状图运行时间从几分钟到一小时不等。

其他实验结果。Fig. 13 显示了 Fig. 5 中列出的基准测试中达到的缓存命中率和最佳缓存命中率。Fig. 12 显示了启用张量并行的 Llama-2-70B 的吞吐量。

D 编译器模式

除了本文主体部分使用的解释器模式外，运行 SGLang 程序的另一种方法是将其编译为计算图，并使用图执行器执行。这为更多编译优化提供了机会，因为我们可以重写图并执行更多静态规划。

D.1 设计与实现

我们为 SGLang 设计了一个中间表示 (IR)，它将 SGLang 程序结构和操作表示为一个计算图。该图包含代表基本运算符的节点和代表依赖关系的边。有关与 Fig. 14a 程序对应的图，请参见 Fig. 14b。在该程序中，每次调用装饰函数或 fork 都会创建一个新的提示状态或流。

节点有多种类型。SGLang 程序中运算符 += 和 + 的每个操作数都由一个 IR 节点表示。这些节点包括 ConstantText、Argument、Gen、Select、Variable、Fork、GetForkItem 和 Join。依赖关系分为两种类型：流内依赖关系，即使用 += 提交到流中的操作必须在该流中所有先前的操作之后执行；以及流间依赖关系，即当一个流需要从另一个流获取变量值时，需要进行同步。诸如 fork 之类的操作会操作多个流，因此会引入流间依赖关系。

```

@function
def expand(s, tip):
    s += (
        "Please expand the following tip into a "
        "detailed paragraph: " + tip + "\n"
    )
    s += gen("paragraph")

@function
def tip_suggestion(s, topic):
    s += "Here are 2 concise tips for " + topic + ".\n"

    # Generate a skeleton of two short tips
    s += "1." + gen("tip_1", stop=["\n", ":", ""]) + "\n"
    s += "2." + gen("tip_2", stop=["\n", ":", ""]) + "\n"

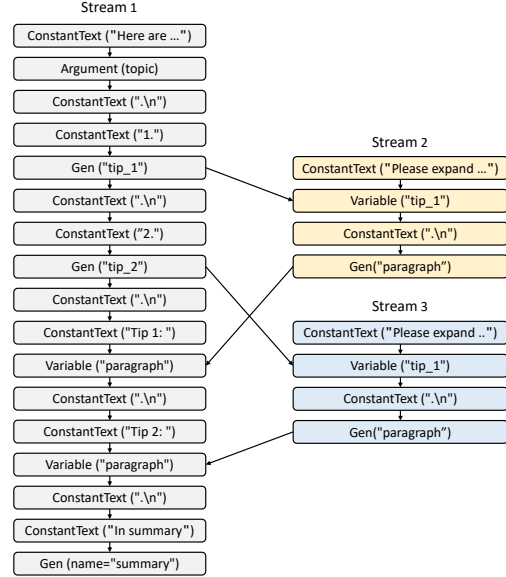
    # Expand the tips into detailed paragraphs in parallel
    detailed_tip1 = expand(tip=s["tip_1"])
    detailed_tip2 = expand(tip=s["tip_2"])

    # Merge the results and generate a summary
    s += "Tip 1: " + detailed_tip1["paragraph"] + "\n"
    s += "Tip 2: " + detailed_tip2["paragraph"] + "\n"
    s += "In summary" + gen("summary")

state = tip_suggestion.run(topic="staying healthy")
print(state.text())

```

(a) SGLang 程序用于并行提示建议，并提供思路框架提示。



(b) Fig. 14a 程序的计算图。三个流分别对应三个函数调用。

Figure 14: SGLang 程序及其对应的数据流图。

为了生成图，我们使用跟踪来运行带有抽象参数的程序，并动态构建图。此方法仅限于不具有数据相关控制流的程序，我们计划在未来的工作中解决这一限制。构建完成后，可以通过图执行器执行图，从而无需重新解释原始 Python 程序。这带来了诸多好处，例如图重写优化、降低运行时开销和程序序列化。执行时，会为每个数据流启动流执行器，并按拓扑顺序将 IR 节点分派到各个流。

D.2 编译器优化案例研究：代码移动以改进前缀共享

我们探索了一种针对 SGLang IR 的编译优化：代码移动以改进前缀共享。我们预计也可以应用更经典的编译器技术，例如自动调优和指令选择。

此优化旨在通过重新排列图中的节点来增加常量前缀的长度，从而改进前缀共享。它并不严格保留原始计算，因此被归类为一种激进的优化。例如，将提示从“这里有一个问题 + { 问题 }。请扮演数学专家并解答给定的问题。”更改为“请扮演数学专家并解答给定的问题。这里有一个问题 + { 问题 }。”可以生成更长的可共享前缀。由于 SGLang 中存在自然语言指令，传统的程序分析方法无法实现这种优化，因此这种优化方式很有意义。我们转而引导 GPT-4 对图节点进行重新排序。我们编写了一个包含多个示例的提示，以教会 GPT-4 SGLang 信息检索的概念，并发现 GPT-4 可以成功地将此优化应用于一些简单的 SGLang 程序。

我们评估了这种优化的有效性。我们从互联网上收集了 20 个提示模板，并在 SGLang 中实现了它们。我们使用其中 5 个模板作为少样本训练示例，其余 15 个作为测试用例。我们的评估表明，对于 15 个模板中的 12 个，GPT-4 成功地对图节点进行了重新排序，并且没有改变语义，这已通过修改后的提示进行人工检查得到证实。平均而言，这种优化使可共享前缀长度增加了 60 个词元，充分展现了 GPT-4 的有效性。创建优化提示符顺序的失败源于对图节点语义理解的偏差。这种方法过于激进，将所有常量都放在前面，即使这种排序方式会改变原始语义。本案例研究旨在探索 GPT-4 在编译器优化中的应用。未来还需要开展更多工作，以确保此类优化的可靠性。